

JOBSHEET 6

INHERITANCE (PEWARISAN)

1. COMPETENCE

- Understand the basic concept of inheritance.
- Capable of creating a subclass of a certain superclass.

2. INTRODUCTION

Inheritance is a way of relegating a more general class to a more specific class. Inheritance is one of the main features of an object-oriented programming language. At the heart of inheritance is the reusable nature of the object-oriented concept. Each subclass will "inherit" the properties of the superclass as long as it is protected or public.

In inheritance, there are two terms that are often used. The class that is descended is called the base class/super class, while the class that is derived is called the derived class (sub class/child class). In Java, declaring a class as a subclass is done by adding the keyword `extends` after the class name declaration, then followed by the name of the parent class---. The `extends` keyword tells the Java compiler that we want to expand the class. Here is an example of an inheritance declaration.

```
public class B extends A {  
...  
}
```

The example above tells the Java compiler that we want to ---extend class A to class B. In other words, class B is a subclass (a child class) of class A, while class A is the parent class of class B.

The characteristics of a superclass are also inherited by its subclasses. There are three types of inheritance: single inheritance, multilevel inheritance, and multiple inheritance. However, this worksheet will focus on single inheritance and multilevel inheritance.

1. Single Inheritance

Single inheritance is a class that has only one parent class. Example:

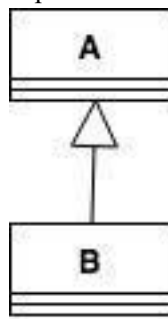


Figure 1. Example of Single Inheritance

Based on Figure 1, it can be seen that class B is a subclass that has one parent, namely class A, so it is called single inheritance.

2. Multilevel Inheritance

Multilevel inheritance is a subclass that can be a superclass for another class.

Example:

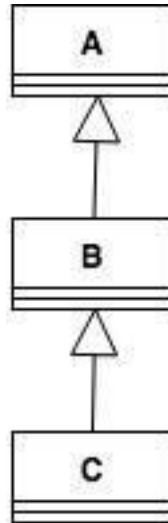


Figure 2. Examples of Multilevel Inheritance

Based on Figure 2 above, it can be seen that class B is a subclass of class A, so in this case class A is a superclass and class B is a subclass. Then class B, which was originally a subclass, has another subclass, namely class C, so that class B becomes a superclass of class C, as well as the opponent if class C has another subclass.

3. Hierarchical Inheritance

Hierarchical inheritance is a type of inheritance in which a class has more than one derived class (subclass). In other words, multiple subclasses share the same parent class. This concept is analogous to a real-world situation in which a parent can have more than one child. An example of hierarchical inheritance can be seen in Figure 3.

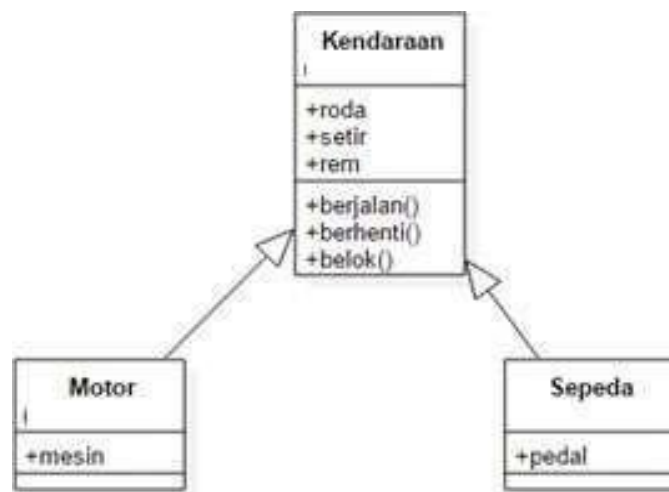


Figure 3 Example of a Class Diagram in Inheritance

Figure 3 illustrates that the Kendaraan class has two subclasses, namely the Motor class and the Sepeda class. Therefore, the Motor and Sepeda classes share the same superclass, and both subclasses may have attributes and behaviors inherited from the Kendaraan class.

4. Hybrid Inheritance

Hybrid inheritance is a type of inheritance that combines two or more types of inheritance. An example of hybrid inheritance is shown in Figure 4.

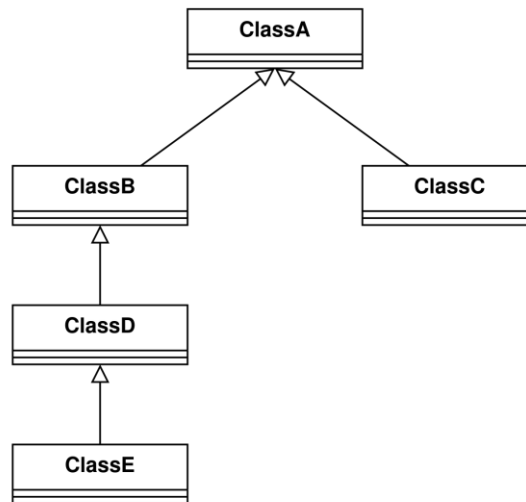


Figure 4. Hybrid Inheritance

As shown in Figure 4, the inheritance structure is a combination of hierarchical and multilevel inheritance. Hierarchical inheritance is represented by Classes A, B, and C, whereas multilevel inheritance is represented by Classes B, D, and E.

There are several rules governing inheritance in the Java programming language, including the following:

1. Multiple inheritance cannot be directly implemented in the Java programming language. Multiple inheritance refers to a situation in which a derived class attempts to extend more than one parent class. An example of multiple inheritance is shown in Figure 5.

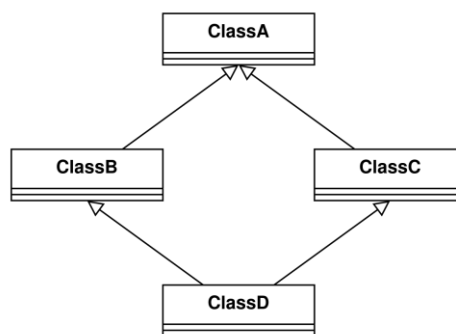
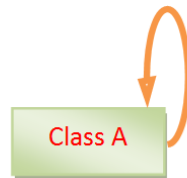


Figure 5. Multiple Inheritance

Suppose there is a `show()` method in both classes B and C, but each has a different implementation. Then, class D extends both classes B and C. When an object of class D attempts to call the `show()` method, the compiler will be unable to determine which method should be executed (the one from class B or the one from class C). This leads to ambiguity.

2. Cyclic inheritance is not allowed in Java. Cyclic inheritance is a type of inheritance in which a class extends itself. This type of inheritance is not permitted in Java because there would be no opportunity to extend the Object class.



3. Attributes and methods with the private access modifier are not inherited.
4. Constructors cannot be inherited. If you want to use the constructor of the superclass, you must use the `super()` statement in the subclass constructor.

In a class diagram, inheritance is represented by a solid line with a triangle at its end. The class closest to the triangle is the superclass, while the class farther from the triangle is the subclass. To create a subclass, the “extends” keyword is used (see the example in the “Inheritance Implementation” session).

A parent class may not pass on some of its members to its subclasses. The extent to which a member can be inherited by another class, or accessed from another class, is closely related to access control. In Java, access control can be illustrated in the following table:

Modifier	class yang sama	package yang sama	subclass	class manapun
private	√			
default	√	√		
protected	√	√	√	
public	√	√	√	√

Super keywords are used to refer to members of the parent class. As the keyword `this` is used to refer to the members of the class itself. The writing format is as follows:

1. `super.namaAtribut`
Referring/accessing attributes from parent class /superclass
2. `super.namaMethod()`
Referring/calling the method from parent class /superclass
3. `super()`
Referring/calling the parent class / superclass constructor can only be used on the first row in the constructor.
4. `super(parameter1, parameter2,dst)`
Referring/calling a parametered constructor of a superclass

When you create an object from a subclass, at that moment the object in the superclass will also be formed. In other words, when the subclass constructor is run to create an object, the superclass constructor will run at that very moment. So in each subclass constructor, on the first row the subclass constructor will be called the superclass constructor. Before a subclass runs its own constructor, it runs a superclass constructor first.

3. EXPERIMENT 1 (extends)

A. STAGES OF EXPERIMENTATION

1. Create a parent/superclass class with the name `ClassA.java`

```

0      *
1      * @author WINDOWS 10
2      */
3      public class ClassA {
4          public int x;
5          public int y;
6
7          public void getNilai() {
8              System.out.println("nilai x:" + x);
9              System.out.println("nilai y:" + y);
10         }
11     }
12

```

2. Create a child class/subclass with the name ClassB.java

```

      * @author WINDOWS 10
      */
      public class ClassB {
          public int z;

          public void getNilaiZ() {
              System.out.println("nilai Z:" + z);
          }

          public void getJumlah() {
              System.out.println("jumlah:" + (x+y+z));
          }
      }

```

3. Create a Percobaan1.java class to run the program above!

```

      *
      */
      public class Percobaan1 {

          /**
           * @param args the command line arguments
           */
          public static void main(String[] args) {

              ClassB hitung = new ClassB();
              hitung.x=20;
              hitung.y=30;
              hitung.z=5;
              hitung.getNilai();
              hitung.getNilaiZ();
              hitung.getJumlah();
          }
      }

```

4. Run the program above, then observe what happens!

B. PERTANYAAN

1. In experiment 1 above, the program that was run ran an error, then fix it so that the program can be run and not error!

.....
.....
.....

2. Explain what caused the program in experiment 1 when there was an error!

.....
.....
.....

4. EXPERIMENT 2 (Access Modifier)

A. STAGES OF EXPERIMENTATION

1. Create a parent/superclass class with the name ClassA.java

```
10  *
11  * @author WINDOWS 10
12  */
13  @
14  public class ClassA {
15      private int x;
16      private int y;
17
18      public void setX(int x){
19          this.x = x;
20      }
21      public void setY(int y){
22          this.y = y;
23      }
24      public void getNilai(){
25          System.out.println("nilai x: "+ x );
26          System.out.println("nilai y: "+ y);
27      }
28  }
```

2. Create a child class/subclass with the name ClassB.java

```
1  * @author WINDOWS 10
2  */
3
4  public class ClassB {
5      private int z;
6
7      public void setZ(int z){
8          this.z = z;
9      }
10     public void getNilaiZ(){
11         System.out.println("nilai Z: "+ z);
12     }
13     public void getJumlah(){
14         System.out.println("jumlah: "+ (x+y+z));
15     }
16 }
```

3. Make a Percobaan2.java class to run the above program!

```

    * @author WINDOWS 10
    */
    public class Percobaan2 {

        /**
         * @param args the command line arguments
         */
        public static void main(String[] args) {

            ClassB hitung = new ClassB();
            hitung.setX(20);
            hitung.setY(30);
            hitung.setZ(5);
            hitung.getNilai();
            hitung.getNilaiZ();
            hitung.getJumlah();

        }

    }

```

4. Run the program above, then observe what happens!

B. QUESTIONS

1. In experiment 2 above, the program ran an error, then fix it so that the program can be run and not error!

.....

.....

.....

2. Explain what caused the program in experiment 1 when there was an error!

.....

.....

.....

5. EXPERIMENT 3 (Super)

A. STAGES OF EXPERIMENTATION

1. Create a parent/superclass class with the name Bangun.java

```

10      * @author WINDOWS 10
11      */
12      public class Bangun {
13          protected double phi;
14          protected int r;
15      }
16
17

```

2. Create a child class/subclass with the name Tabung.java

```

11  L  */
12  public class Tabung extends Bangun{
13      protected int t;
14      public void setSuperPhi(double phi){
15          super.phi = phi;
16      }
17      public void setSuperR(int r){
18          super.r = r;
19      }
20      public void setT(int t){
21          this.t = t;
22      }
23
24      public void volume(){
25          System.out.println("Volume Tabung adalah: "+(super.phi*super.r*super.r*this.t));
26      }
27  }
28  }
29

```

3. Make a Percobaan3.java class to run the above program!

```

11  L  */
12  public class Percobaan3 {
13
14      /**
15       * @param args the command line arguments
16       */
17      public static void main(String[] args) {
18          // TODO code application logic here
19          Tabung tabung=new Tabung();
20          tabung.setSuperPhi(3.14);
21          tabung.setSuperR(10);
22          tabung.setT(3);
23          tabung.volume();
24      }
25  }
26

```

4. Run the program above!

```

Output - Percobaan3 (run)

run:
Volume Tabung adalah: 942.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

B. QUESTIONS

1. Describe the "super" function in the following program pieces in the Tubes class!

```

public void setSuperPhi(double phi){
    super.phi = phi;
}

public void setSuperR(int r){
    super.r = r;
}

```


-
-
-
2. Explain the functions of "super" and "this" in the following program snippets in the Tube!

```
public void volume() {  
    System.out.println("Volume Tabung adalah: " + (super.phi*super.r*super.r*this.t));  
}
```

-
-
-
3. Explain why the Tube class doesn't declare the "phi" and "r" attributes, but the class can access them!
-
-
-

6. EXPERIMENT 4 (super constructor)

A. STAGES OF EXPERIMENTATION

1. Create three files with the names ClassA.java, ClassB.java, and ClassC.java, as shown in the program code below!

ClassA.java

```
10 | * @author WINDOWS 10  
11 | */  
12 |  
13 | public class ClassA {  
14 |     ClassA() {  
15 |         System.out.println("konstruktor A dijalankan");  
16 |     }  
17 | }
```

ClassB.java

```
9 | *  
10 | * @author WINDOWS 10  
11 | */  
12 |  
13 | public class ClassB extends ClassA {  
14 |     ClassB() {  
15 |         System.out.println("konstruktor B dijalankan");  
16 |     }  
17 | }
```

ClassC.java

```

9      *
10     * @author WINDOWS 10
11     */
12     public class ClassC extends ClassB{
13     ClassC(){
14         System.out.println("konstruktor C dijalankan");
15     }
16     }
17

```

1. Create a Percobaan4.java class to run the above program!

```

10     * @author WINDOWS 10
11     */
12     public class Percobaan4 {
13
14     /**
15      * @param args the command line arguments
16      */
17     public static void main(String[] args) {
18         // TODO code application logic here
19         ClassC test= new ClassC();
20     }
21
22     }
23

```

2. Run the program then observe what happens!

```

: Output - Percobaan4 (run)

run:
konstruktor A dijalankan
konstruktor B dijalankan
konstruktor C dijalankan
BUILD SUCCESSFUL (total time: 0 seconds)

```

B. QUESTIONS

1. In experiment 4, specify which class belongs to superclasses and subclasses, then explain why!

.....

.....

.....

2. Change the contents of the default ClassC constructor as follows:

```

public class ClassC extends ClassB{
    ClassC(){
        super();
        System.out.println("konstruktor C dijalankan");
    }
}

```

Add the word `super()` in the Garden row in its default constructor. Try running the Experiment4 class again and there is no difference in the output!

- The contents of the default ClassC constructor can be as follows:

```

12 public class ClassC extends ClassB{
13     ClassC() {
14         System.out.println("konstruktor C dijalankan");
15         super();
16     }
17 }

```

When changing the position of `super()` in the second row in its default constructor there is an error. Then return the first line `super()` as before, then the error will disappear.

Notice the output results when the Experiment4 class is run. Why can the output be displayed as follows when the test object is institutionalized from the ClassC class

```

: Output - Percobaan4 (run)

run:
konstruktor A dijalankan
konstruktor B dijalankan
konstruktor C dijalankan
BUILD SUCCESSFUL (total time: 0 seconds)

```

Explain how the constructor runs when the test object is created!

.....

.....

.....

- What is the `super()` function in the below program snippet in ClassC!

```

public class ClassC extends ClassB{
    ClassC() {
        super();
        System.out.println("konstruktor C dijalankan");
    }
}

```

.....

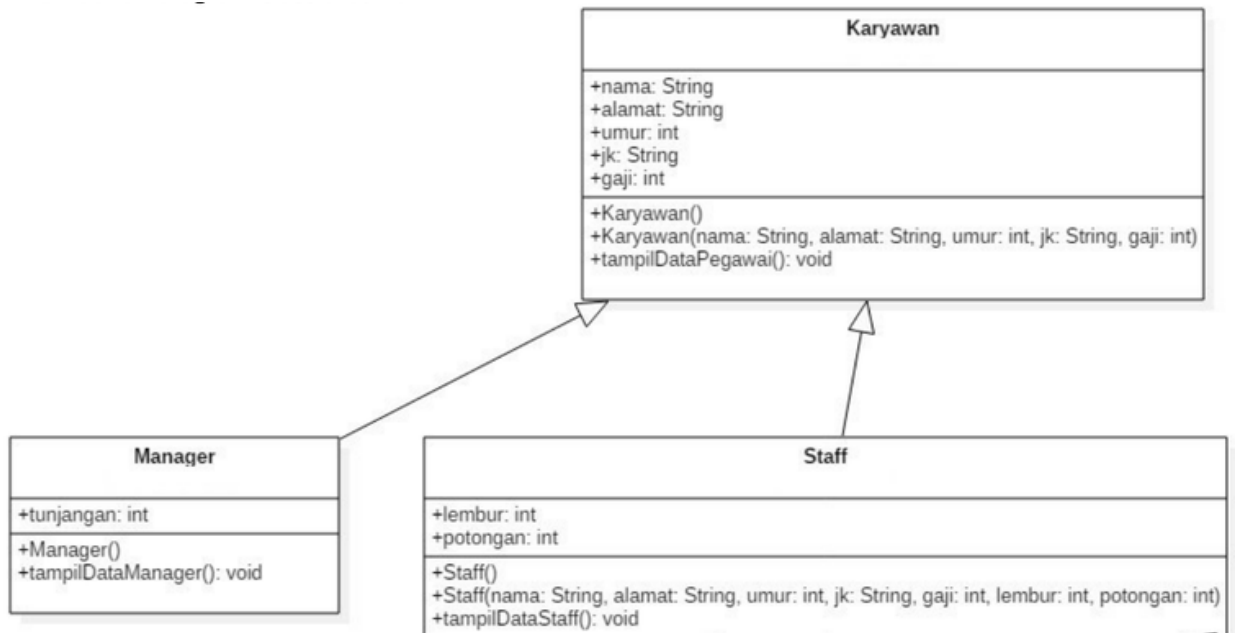
.....

.....

7. EXPERIMENT 5

A. STAGES OF EXPERIMENTATION

Consider the following class diagram:



1. Make Karyawan class

```

13 public class Karyawan {
14     public String nama, alamat, jk;
15     public int umur, gaji;
16
17     public Karyawan() {
18     }
19
20     public Karyawan(String nama, String alamat, String jk, int umur, int gaji) {
21         this.nama=nama;
22         this.alamat=alamat;
23         this.jk=jk;
24         this.umur=umur;
25         this.gaji=gaji;
26     }
27
28     public void tampilDataKaryawan() {
29         System.out.println("Nama      =" +nama);
30         System.out.println("Alamat   =" +alamat);
31         System.out.println("Jenis Kelamin =" +jk);
32         System.out.println("Umur     =" +umur);
33         System.out.println("Gaji     =" +gaji);
34     }
35 }
  
```

2. Make Manager class

```

12 public class Manager extends Karyawan{
13     public int tunjangan;
14
15     public Manager() {
16
17     }
18     public void tampilDataManager(){
19         super.tampilDataKaryawan();
20         System.out.println("Tunjangan          =" +tunjangan);
21         System.out.println("Total Gaji          =" +(super.gaji+tunjangan));
22     }
23
24 }

```

3. Make Staff class

```

13 public class Staff extends Karyawan {
14     public int lembur, potongan;
15
16     public Staff(){
17
18     }
19     public Staff(String nama, String alamat, String jk, int umur, int gaji, int lembur, int potongan){
20         super(nama,alamat,jk,umur,gaji);
21         this.lembur=lembur;
22         this.potongan=potongan;
23     }
24     public void tampilDataStaff(){
25         super.tampilDataKaryawan();
26         System.out.println("Lembur          =" +lembur);
27         System.out.println("Potongan        =" +potongan);
28         System.out.println("Total Gaji      =" +(gaji+lembur-potongan));
29     }

```

4. Make Inheritance1 class

```

12 public class Inheritance1 {
13
14     /**
15      * @param args the command line arguments
16      */
17     public static void main(String[] args) {
18         // TODO code application logic here
19         Manager M = new Manager();
20         M.nama="Vivin";
21         M.alamat="Jl. Vinolia";
22         M.umur=25;
23         M.jk="Perempuan";
24         M.gaji=3000000;
25         M.tunjangan=1000000;
26         M.tampilDataManager();
27
28         Staff S = new Staff();
29         S.nama="Lestari";
30         S.alamat="Malang";
31         S.umur=25;
32         S.jk="Perempuan";
33         S.gaji=2000000;
34         S.lembur=500000;
35         S.potongan=250000;
36         S.tampilDataStaff();
37     }
38
39 }

```

5. Run the program. The output will be displayed as follows:


```
Output - inheritance1 (run)

run:
Nama      =Vivin
Alamat    =Jl. Vinolia
Jenis Kelamin =Perempuan
Umur      =25
Gaji      =3000000
Tunjangan =1000000
Total Gaji =4000000
Nama      =Lestari
Alamat    =Malang
Jenis Kelamin =Perempuan
Umur      =25
Gaji      =2000000
Lembur    =500000
Potongan  =250000
Total Gaji =2250000
BUILD SUCCESSFUL (total time: 0 seconds)
```

B. QUESTIONS

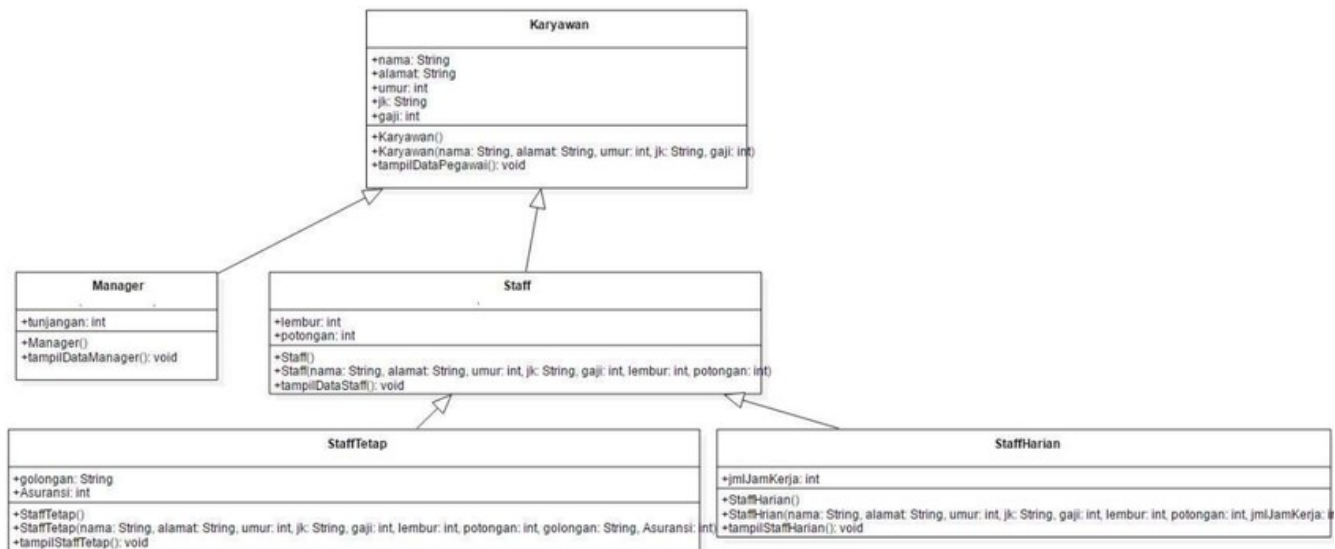
1. Identify which classes are the superclass and subclass in Experiment 1 above!
2. What keyword is used to derive one class from another class?
3. Examine the program code in the Manager class. What attributes does the class have? Identify which attributes are inherited from the Karyawan class!
4. Explain the use of the `super` keyword in the code snippet below, which appears in the Manager class!

```
System.out.println("Total Gaji      =" + (super.gaji+tunjangan));
```
5. What type of inheritance is demonstrated in Experiment 1 above? Explain your answer!

8. EXPERIMENT 6

A. STAGES OF EXPERIMENTATION

Consider the following class diagram:



1. Based on the program you created in Experiment 1, add two classes, namely the StaffTetap class and the StaffHarian class. The program code is as follows:

```

12  public class StaffTetap extends Staff{
13      public String golongan;
14      public int asuransi;
15
16      public StaffTetap() {
17
18      }
19      public StaffTetap(String nama, String alamat, String jk, int umur,
20          int gaji, int lembur, int potongan,String golongan, int asuransi){
21          super(nama,alamat,jk,umur,gaji, potongan, lembur);
22          this.golongan=golongan;
23          this.asuransi=asuransi;
24      }
25      public void tampilStaffTetap(){
26          System.out.println("-----Data Staff Tetap-----");
27          super.tampilDataStaff();
28          System.out.println("Golongan      =" +golongan);
29          System.out.println("Jumlah Asuransi =" +asuransi);
30          System.out.println("Gaji Bersih      =" +(gaji+lembur-potongan-asuransi));
31      }
32
33  }
  
```

2. Make StaffHarian.java class


```

12 public class StaffHarian extends Staff{
13     public int jmlJamKerja;
14
15     public StaffHarian(){
16
17     }
18     public StaffHarian(String nama, String alamat, String jk, int umur,
19         int gaji, int lembur, int potongan,int jmlJamKerja){
20         super(nama,alamat,jk,umur,gaji,lembur,potongan);
21         this.jmlJamKerja=jmlJamKerja;
22     }
23     public void tampilStaffHarian(){
24         System.out.println("=====Data Staff Harian=====");
25         super.tampilDataStaff();
26         System.out.println("Jumlah Jam Kerja =" +jmlJamKerja);
27         System.out.println("Gaji Bersih      =" +(gaji*jmlJamKerja+lembur-potongan));
28     }
29
30 }

```

After creating the two classes above, edit the inheritance1.java class as follows:

```

12 public class Inheritance1 {
13
14     /**
15      * @param args the command line arguments
16      */
17     public static void main(String[] args) {
18         // TODO code application logic here
19         StaffTetap ST= new StaffTetap("Budi", "Malang", "Lakilaki", 20, 2000000, 250000, 200000, "2A", 100000);
20         ST.tampilStaffTetap();
21
22
23         StaffHarian SH = new StaffHarian("Indah", "Malang", "Perempuan", 27, 10000, 100000, 50000, 100);
24         SH.tampilStaffHarian();
25     }
26
27 }

```

Run the program. The output should be displayed as follows:

```

Output - inheritance1 (run)

run:
=====Data Staff Tetap=====
Nama           =Budi
Alamat         =Malang
Jenis Kelamin  =Lakilaki
Umur           =20
Gaji           =2000000
Lembur         =200000
Potongan       =250000
Total Gaji     =1950000
Golongan       =2A
Jumlah Asuransi=100000
Gaji Bersih    =1850000
=====Data Staff Harian=====
Nama           =Indah
Alamat         =Malang
Jenis Kelamin  =Perempuan
Umur           =27
Gaji           =10000
Lembur         =100000
Potongan       =50000
Total Gaji     =60000
Jumlah Jam Kerja=100
Gaji Bersih    =1050000
BUILD SUCCESSFUL (total time: 0 seconds)

```

B. QUESTIONS

1. Based on the classes above, which one represents single inheritance and which one represents multilevel inheritance?
2. Examine the program code for the StaffTetap and StaffHarian classes. What attributes do these classes have? Identify which attributes are inherited from the Staff class!
3. What is the purpose of the following code snippet in the StaffHarian class?

```
super(nama, alamat, jk, umur, gaji, lembur, potongan);
```

4. What is the purpose of the following code snippet in the StaffHarian class?

```
super.tampilDataStaff();
```

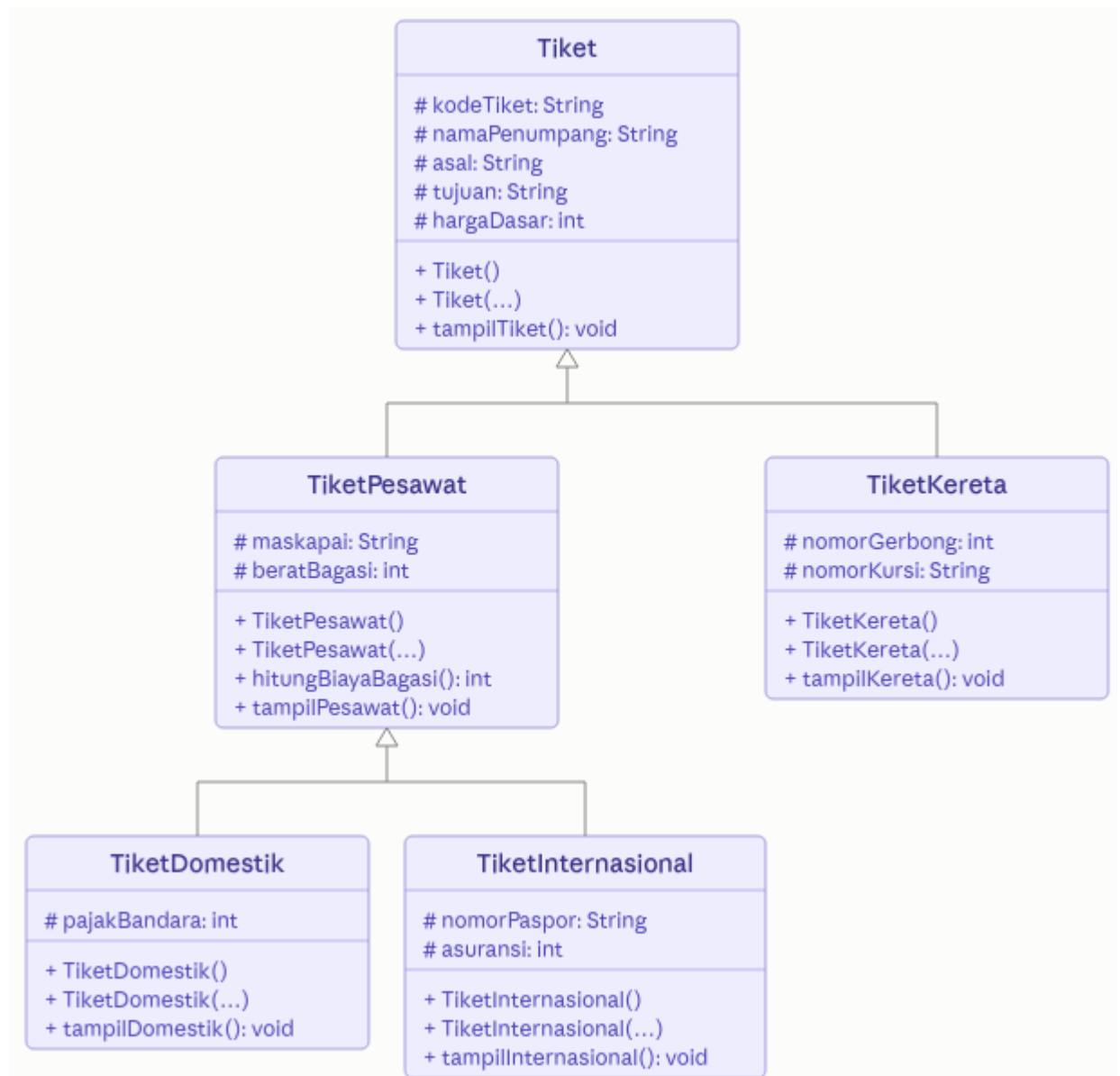
5. Examine the following program code found in the StaffTetap class

```
System.out.println("Gaji Bersih      =" + (gaji + lembur - potongan - asuransi));
```

It can be seen from the code snippet above that the salary, overtime, and deduction attributes can be accessed directly. Why is this possible, and how does the StaffTetap class have the salary, overtime, and deduction attributes even though these attributes are not declared in the class itself?

9. EXERCISE

Observe the following class diagram:



1. Create a program that implements the **inheritance concept** based on the class diagram above. Then, create a `TestTiket` class to display data for **train tickets**, **domestic flight tickets**, and **international flight tickets**.

Order of parameters for parameterized constructors:

- `Tiket(kodeTiket, namaPenumpang, asal, tujuan, hargaDasar)`
- `TiketKereta( Tiket parameters , nomorGerbong, nomorKursi)`
- `TiketPesawat( Tiket parameters , maskapai, beratBagasi)`
- `TiketDomestik( TiketPesawat parameters , pajakBandara)`
- `TiketInternasional( TiketPesawat parameters , nomorPaspor, asuransi)`

Requirements:

- Every parameterized constructor **must call its parent constructor using** `super(...)`.
- Every display method must call the display method of its parent class. For example, `tampilDomestik()` must call `super.tampilPesawat()`.

- `hitungBiayaBagasi()`: The first 20 kg of baggage is free. Any excess baggage is charged at Rp50,000 per kg.
- The total payment for each ticket type is calculated as follows:
 - Train = `hargaDasar`
 - Domestic = `hargaDasar + hitungBiayaBagasi() + pajakBandara`
 - International = `hargaDasar + hitungBiayaBagasi() + asuransi`
- At least one object must be created using the no-argument constructor, and its attributes must then be assigned one by one, as demonstrated in Experiment 1.

Test Data and Expected Output:

```

===== Tiket Kereta =====
Kode Tiket      = KA-001
Nama Penumpang  = Andi
Rute            = Malang - Jakarta
Harga Dasar     = 350000
Nomor Gerbong   = 3
Nomor Kursi     = 12A
Total Bayar     = 350000
===== Tiket Pesawat Domestik =====
Kode Tiket      = GA-102
Nama Penumpang  = Sinta
Rute            = Surabaya - Denpasar
Harga Dasar     = 900000
Maskapai        = Garuda Indonesia
Berat Bagasi    = 25 kg
Biaya Bagasi    = 250000
Pajak Bandara   = 75000
Total Bayar     = 1225000
===== Tiket Pesawat Internasional =====
Kode Tiket      = SQ-205
Nama Penumpang  = Budi
Rute            = Jakarta - Singapura
Harga Dasar     = 2500000
Maskapai        = Singapore Airlines
Berat Bagasi    = 20 kg
Biaya Bagasi    = 0
Nomor Paspor    = C1234567
Asuransi        = 150000
Total Bayar     = 2650000

```

2. Answer the following questions based on the program you have created:

- a. What type of inheritance does the program implement? Identify which parts represent **hierarchical inheritance** and which parts represent **multilevel inheritance**.
- b. List all attributes possessed by a `TiketInternasional` object, along with the class from which each attribute originates.
- c. Why can `TiketDomestik` call `hitungBiayaBagasi()` even though the method is not declared directly inside the class?
- d. Change the access modifier of `hargaDasar` in the `Tiket` class to `private`, and then compile the program again. What happens? Provide **one way to fix the problem without changing the modifier back to `protected`**.